

HTML5 Exercises — Module 1

Theme: Local Community Event Portal

Exercise 1: Create the HTML5 Base Template

index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Local Community Event Portal</title>
</head>
<body>

  <!-- Navigation -->
  <nav> ... </nav>

  <!-- Main -->
  <section id="home"> ... </section>
  <section id="events"> ... </section>
  <section id="contact"> ... </section>

  <!-- Footer -->
  <footer> ... </footer>

</body>
</html>
```

Exercise 2: Navigation and Linking

index.html:

```
<!-- Nav with anchor links to sections -->
<nav>
  <a href="#home">Home</a> |
  <a href="#events">Events</a> |
  <a href="#contact">Contact</a> |
  <a href="help.html" target="_blank">Help</a>
</nav>

<!-- Matching section IDs -->
<section id="home"> ... </section>
<section id="events"> ... </section>
<section id="contact"> ... </section>
```

Exercise 3: Welcome Message with Styling and ID/Class

Internal CSS in :

```
<style>
  /* Blue background for welcome banner */
  #welcomeBanner {
    background-color: blue;
    color: white;
    padding: 15px;
  }

  /* Highlight class */
  .highlight {
    background-color: yellow;
    font-weight: bold;
  }
</style>
```

index.html — welcome banner with inline span and class:

```
<div id="welcomeBanner">
  <h1>Welcome to the Community Event Portal</h1>
  <!-- Inline style: red, bold -->
  <p>Special Offer:
    <span style="color: red; font-weight: bold;">
      Free entry for first 50 registrations!
    </span>
  </p>
  <!-- highlight class -->
  <p class="highlight">Check out the latest events in your city.</p>
</div>
```

Exercise 4: Image Gallery for Community Events

Internal CSS:

```
.event-img {
  border: 2px solid black;
  width: 150px;
  height: 100px;
}
```

index.html — 2 rows x 3 columns table with images:

```
<table border="1" cellpadding="5">
  <caption>Community Events Photo Gallery</caption>
  <tr>
    <td>
      
      <p>Clean-Up Drive</p>
    </td>
    <td>
      
      <p>Cultural Fest</p>
    </td>
    <td>
      
      <p>Health Camp</p>
    </td>
  </tr>
  <tr>
    <td>
      
      <p>Book Fair</p>
    </td>
    <td>
      
      <p>Marathon</p>
    </td>
    <td>
      
      <p>Science Olympiad</p>
    </td>
  </tr>
</table>
```

Exercise 5: Event Registration Form

Internal CSS for form:

```

form {
  display: flex;
  flex-direction: column;
  max-width: 400px;
  gap: 8px;
}
form input, form select, form textarea {
  padding: 6px;
  font-size: 1rem;
}
form button {
  padding: 8px;
  background-color: #007BFF;
  color: white;
  border: none;
  cursor: pointer;
}

```

index.html — registration form:

```

<form id="registrationForm" onsubmit="showConfirmation(event)">

  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name"
    placeholder="Enter your full name"
    required autofocus />

  <label for="email">Email:</label>
  <input type="email" id="email" name="email"
    placeholder="you@example.com" required />

  <label for="eventDate">Preferred Date:</label>
  <input type="date" id="eventDate" name="eventDate" required />

  <label for="eventType">Event Type:</label>
  <select id="eventType" name="eventType" required
    onchange="showFee(this.value)">
    <option value="">-- Select Event --</option>
    <option value="cleanup">Community Clean-Up (Free)</option>
    <option value="cultfest">Cultural Fest (Rs. 50)</option>
    <option value="healthcamp">Health Camp (Free)</option>
    <option value="marathon">Marathon (Rs. 100)</option>
  </select>

  <label for="message">Message:</label>
  <textarea id="message" name="message" rows="4"
    placeholder="Write your message..."></textarea>

  <button type="submit">Register</button>
</form>

<!-- output element for confirmation message -->
<output id="confirmationOutput"></output>

```

JavaScript — showConfirmation():

```

function showConfirmation(event) {
  event.preventDefault();
  var name = document.getElementById("name").value;
  document.getElementById("confirmationOutput").value =
    "Thank you, " + name + "! Your registration is confirmed.";
}

```

Exercise 6: Event Feedback with Event Handling

onblur — validate phone number:

```

<input type="text" id="phone" placeholder="10-digit number"
      onblur="validatePhone(this.value)" />
<span id="phoneError" style="color:red;"></span>

function validatePhone(value) {
  var errorEl = document.getElementById("phoneError");
  if (!/^d{10}$/.test(value)) {
    errorEl.textContent = "Please enter a valid 10-digit phone number.";
  } else {
    errorEl.textContent = "";
  }
}

```

onchange — show event fee:

```

<select id="eventType" onchange="showFee(this.value)"> ... </select>
<span id="feeDisplay"></span>

var fees = {
  cleanup:    "Entry: Free",
  cultfest:   "Entry Fee: Rs. 50",
  healthcamp: "Entry: Free",
  marathon:   "Entry Fee: Rs. 100"
};

function showFee(value) {
  document.getElementById("feeDisplay").textContent = fees[value] || "";
}

```

onclick — confirmation on submit button:

```

<button type="submit" onclick="return true;">Register</button>

```

ondblclick — enlarge image:

```



var enlarged = false;
function enlargeImage() {
  var img = document.getElementById("promoImg");
  if (!enlarged) {
    img.style.width = "400px";
    enlarged = true;
  } else {
    img.style.width = "200px";
    enlarged = false;
  }
}

```

onkeyup — character count in textarea:

```

<textarea id="message" onkeyup="countChars(this.value)"></textarea>
<span id="charCount">Characters: 0</span>

function countChars(value) {
  document.getElementById("charCount").textContent =
    "Characters: " + value.length;
}

```

Exercise 7: Video Invite with Media Events

index.html — video element with oncanplay:

```

<video id="promoVideo" width="400" controls
      oncanplay="videoReady()">
  <source src="promo.mp4" type="video/mp4" />
  Your browser does not support the video tag.
</video>

```

```
<p id="videoStatus"></p>
```

```
function videoReady() {  
    document.getElementById("videoStatus").textContent =  
        "Video ready to play!";  
}
```

onbeforeunload — warn user before leaving page:

```
window.onbeforeunload = function () {  
    return "You have unsaved changes. Are you sure you want to leave?";  
};
```

Exercise 8: Saving User Preferences

Save to localStorage on form submit:

```
// Save  
var eventType = document.getElementById("eventType").value;  
localStorage.setItem("preferredEvent", eventType);
```

Retrieve on page load and pre-select:

```
window.onload = function () {  
    var saved = localStorage.getItem("preferredEvent");  
    if (saved) {  
        document.getElementById("savedPref").textContent = saved;  
        document.getElementById("eventType").value = saved;  
    }  
};
```

Clear both localStorage and sessionStorage:

```
<button onclick="clearPreferences()">Clear Preferences</button>  
  
function clearPreferences() {  
    localStorage.clear();  
    sessionStorage.clear();  
    document.getElementById("savedPref").textContent = "None";  
    alert("Preferences cleared.");  
}
```

Exercise 9: Geolocation for Event Mapping

index.html + JavaScript:

```
<button onclick="findNearbyEvents()">Find Nearby Events</button>  
<p id="locationOutput"></p>  
  
function findNearbyEvents() {  
    var output = document.getElementById("locationOutput");  
  
    if (!navigator.geolocation) {  
        output.textContent = "Geolocation is not supported by your browser.";  
        return;  
    }  
  
    output.textContent = "Locating...";  
  
    navigator.geolocation.getCurrentPosition(  
        // Success  
        function (position) {  
            var lat = position.coords.latitude;  
            var lon = position.coords.longitude;  
            output.textContent =  
                "Latitude: " + lat.toFixed(4) + ", Longitude: " + lon.toFixed(4);  
        },  
        // Error handling  
        function (error) {
```

```

switch (error.code) {
  case error.PERMISSION_DENIED:
    output.textContent = "Permission denied.";
    break;
  case error.POSITION_UNAVAILABLE:
    output.textContent = "Location unavailable.";
    break;
  case error.TIMEOUT:
    output.textContent = "Request timed out.";
    break;
  default:
    output.textContent = "An unknown error occurred.";
}
},
// Options: high accuracy, timeout
{
  enableHighAccuracy: true,
  timeout: 10000,
  maximumAge: 0
}
);
}

```

Exercise 10: Debugging with Chrome DevTools

No code required — browser-based debugging task.

1. Right-click any element on the page and click Inspect to open DevTools.
2. Elements tab — click any element and edit its CSS live in the Styles panel.
3. Console tab — view logs from the script. These `console.log()` calls are already in `index.html`:

```

console.log("Portal page loaded.");
console.log("localStorage contents:", localStorage);

```

4. Sources tab — open `index.html`, click a line number to add a breakpoint, then reload the page to step through JS execution and watch variable values.
5. Network tab — verify all files (CSS, images, video) load with status 200.